

CECS 440 – Lab 10

MIPS Execute Stage

Due date: 12/2/2025

Name: Eric Santana

ID: 015107467

I certify that this submission is my original work.



Goal

The goal of this lab was to implement the Execute stage of the MIPS processor. This stage is responsible for performing the actual arithmetic and logical operations, selecting the correct ALU inputs, choosing the destination register, and calculating the branch target address.

Another goal for this lab was to make sure all the control signals passed from the Decode stage into the Execute stage were forwarded correctly, without being changed. Finally, we needed to verify everything using the given test bench in Vivado.

Steps

- Started a new Vivado project for Lab 10.
- Added the required modules from previous labs (mux2.v and adder32.v) and created the new files: alu.v, sl2.v, and EX_Stage.v.
- Wrote and completed the Verilog code for each module according to the lab instructions.
- Added the testbench and ran the simulation.
- Fixed the errors related to missing modules by importing the mux and adder from earlier labs.
- Reran the simulation and checked the console output to confirm everything matched the expected behavior.

Results

The simulation showed that the Execute stage worked the way it was supposed to. Each ALU operation produced the correct output based on the ALUControl setting, and the Zero flag matched the ALU result every time. The testbench checked all of these automatically, and no errors were printed. It also checked that the control signals RegWrite, MemtoReg, MemWrite, Branch passed straight through the Execute stage without being modified. Once I added the missing mux and adder files, the Execute stage passed all the tests.

While reviewing the waveform and console messages, I also confirmed the role of the Execute stage for different instruction types. For R-type instructions, the Execute stage handled the arithmetic and logical operations and selected the correct destination register using the RegDst signal. For load and store instructions, the Execute stage calculated the effective memory address by adding the base register to the sign extended immediate, which the ALUSrc mux selected correctly. For branch instructions like the BEQ, the Execute stage used the ALU to compare the two register values and set the Zero flag, and it also computed the branch target address using the shift left 2 unit and the PCPlus4 adder.

The simulation also helped verify the function of the control signals in the Execute stage. ALUControl chose the ALU operation, ALUSrc picked between the register file and the immediate value, and RegDst determined which register would be written for R-type instructions. The other signals RegWrite, MemWrite, MemtoReg, Branch were forwarded unchanged so the later pipeline stages could make the correct decisions.

Overall, the results matched exactly what the lab expected, and the Execute stage behaved correctly for all instruction types.

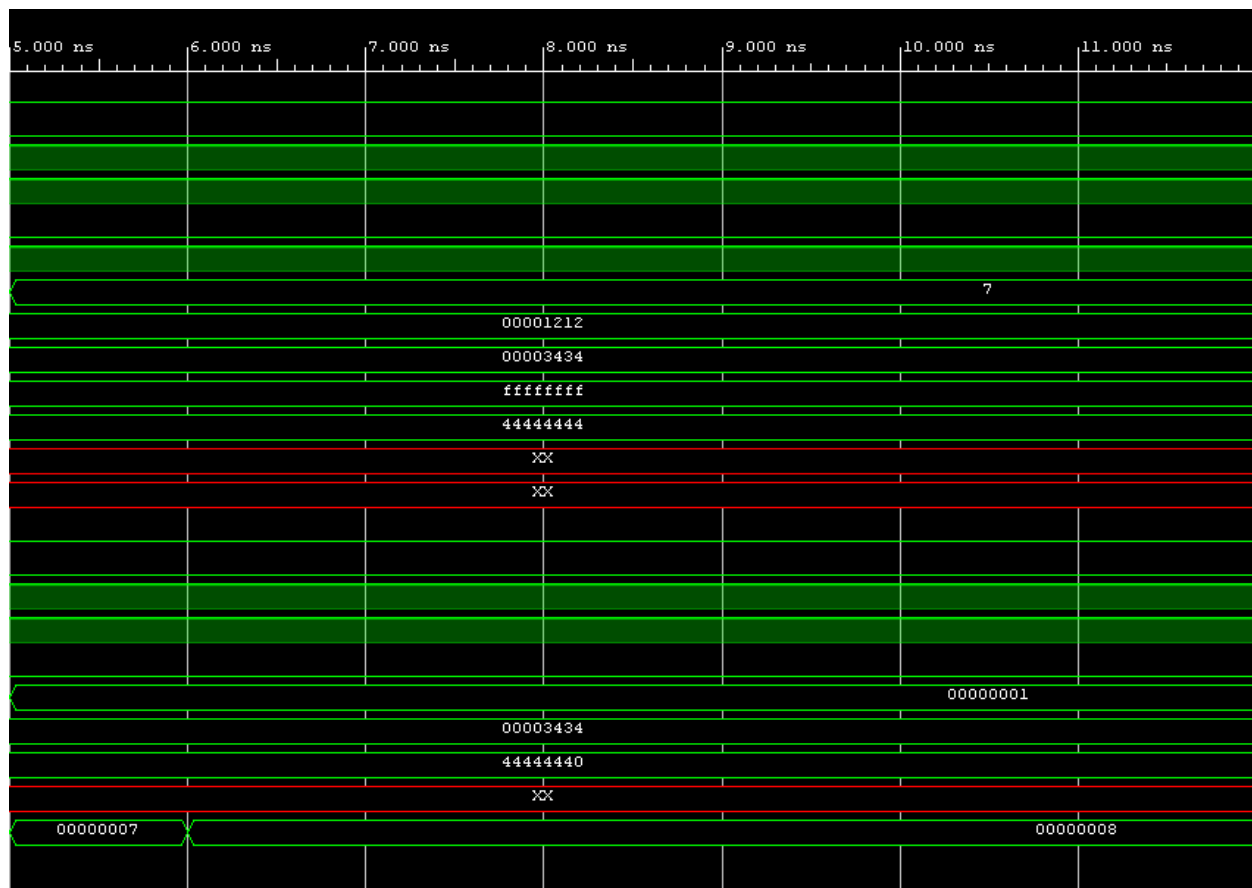
Conclusion

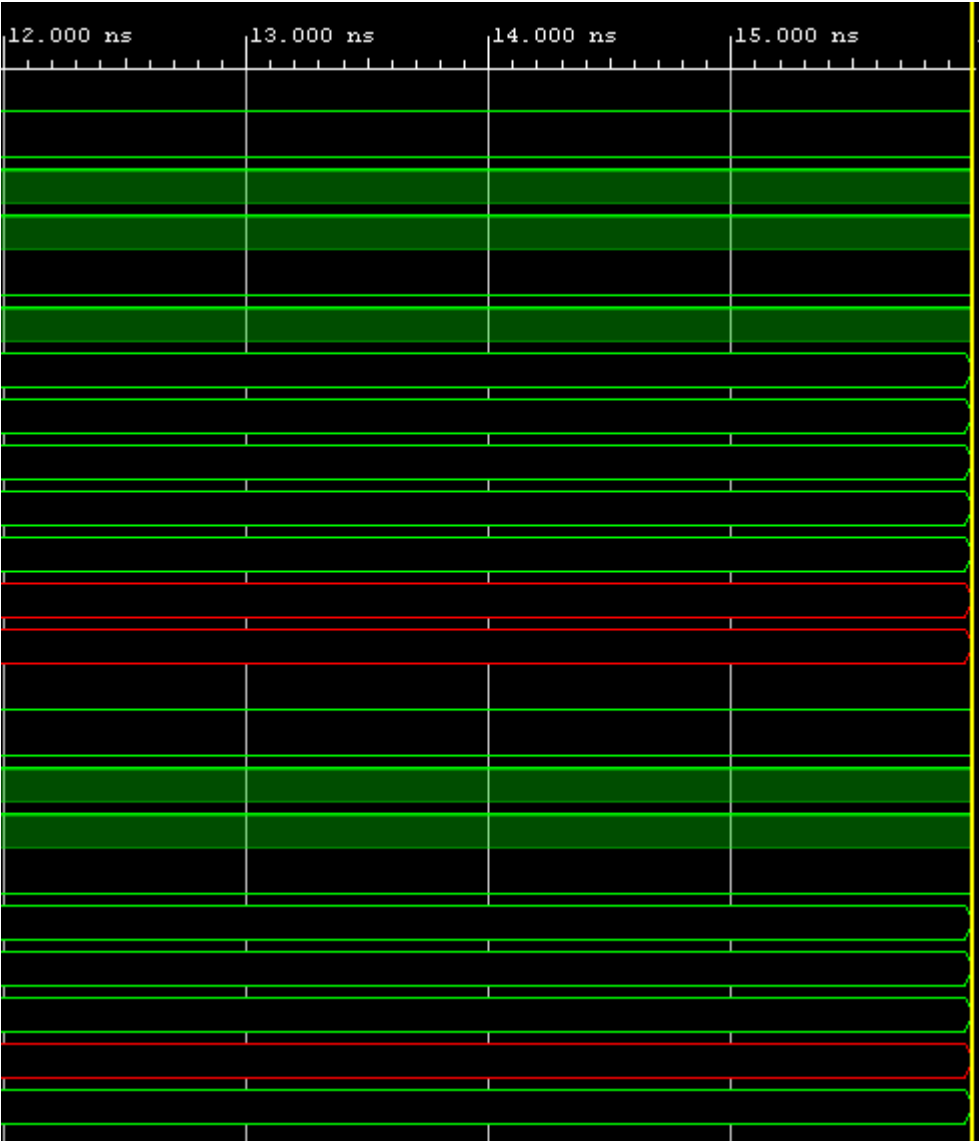
This lab helped me understand how the Execute stage fits into the overall MIPS processor. The lab also showed how important it is to wire the multiplexers correctly so the ALU gets the right inputs and the correct destination register is chosen. Another big lesson was making sure all the modules from earlier labs are included, since missing files can stop the whole design from running. After fixing those issues, everything worked and the Execute stage behaved exactly like it should.

Appendix (1 out of 3)

Waveform







Appendix (2 out of 3)

Console Output

```
# }
# run 1000ns
ALU is performing AND
aluoutE = 00001010
ALU is performing OR
aluoutE = 00003636
ALU is performing ADD
aluoutE = 00004646
ALU Operation is invalid
aluoutE = xxxxxxxx
ALU is performing SUB
aluoutE = ffffddde
ALU is performing SLT
aluoutE = 00000001
$finish called at time : 16 ns : File "C:/Users/fallo/Desktop/CECS 440/Lab 10
INFO: [USF-XSim-96] XSim completed. Design snapshot 't_EX_Stage_behav' loaded
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:04 ; elapsed = 00:00:06 . Memory (MB
```

Appendix (3 out of 3)

Code

Alu.v

```
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////  
timescale 1ns / 1ps  
// Lab 10  
// Engineer: Eric Santana  
// ID: 015107467  
module alu(srca, srcb, alucontrol, aluout, zero);  
    input  [31:0] srca, srcb;  
    input  [2:0]  alucontrol;  
    output [31:0] aluout;  
    output      zero;  
  
    assign aluout = (alucontrol == 0) ? (srca & srcb) : // AND  
                    (alucontrol == 1) ? (srca | srcb) : // OR  
                    (alucontrol == 2) ? (srca + srcb) : // ADD  
                    (alucontrol == 6) ? (srca - srcb) : // SUB  
                    (alucontrol == 7) ? (srca < srcb) : // SLT  
                    32'hx; // default  
  
    assign zero = (aluout == 0);  
  
endmodule
```

EX_stage.v

```
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
`timescale 1ns / 1ps
// Lab 10
// Engineer: Eric Santana
// ID: 018107467

module EX_Stage(
    regwriteD,
    memtooregD,
    memwriteD,
    branchD,
    alucontrolE,
    alusrcE,
    regdstE,
    srcsE,
    writedataD,
    rxE,
    rdE,
    signimmE,
    pcplus4E,

    regwriteE,
    memtooregE,
    memwriteE,
    branchE,
    zeroE,
    aluoutE,
    writedataE,
    writeregE,
    pcbranchE
);

    input      regwriteD, memtooregD, memwriteD, branchD;
    input [2:0] alucontrolE;
    input      alusrcE, regdstE;
    input [31:0] srcsE, writedataD, signimmE, pcplus4E;
    input [4:0] rxE, rdE;

    output      regwriteE, memtooregE, memwriteE, branchE, zeroE;
    output [31:0] aluoutE, writedataE, pcbranchE;
    output [4:0] writeregE;

    wire [31:0] srcbE, signimmhE;

    assign regwriteE = regwriteD;
    assign memtooregE = memtooregD;
    assign memwriteE = memwriteD;
    assign branchE   = branchD;
    assign writedataE = writedataD;

    alu alu(
        .srcs(srcsE),
        .srcb(srcbE),
        .alucontrol(alucontrolE),
        .aluout(aluoutE),
        .zero(zeroE)
    );

    mux2 #32 srcbmux(
        .d0(writedataD),
        .d1(signimmE),
        .s(srcbE),
        .y(srcbE)
    );

    mux2 #5 wrmux(
        .d0(rxE),
        .d1(rdE),
        .s(regdstE),
        .y(writeregE)
    );

    s12 immh(
        .a(signimmE),
        .y(signimmhE)
    );

    adder32 pcadd3(
        .a(pcplus4E),
        .b(signimmhE),
        .y(pcbranchE)
    );

endmodule
```


SI2.v

```
timescale 1ns / 1ps
// Lab 10
// Engineer: Eric Santana
// ID: 015107467

module sl2(a, y);
    input  [31:0] a;
    output [31:0] y;

    // shift left by 2 (append two zeros)
    assign y = {a[29:0], 2'b00};
endmodule
```

T_EX_Stage.v

```
////////////////////////////////////
`timescale 1ns / 1ps
// Lab 10
// Engineer: Eric Santana
// ID: 015107467
//
module t_EX_Stage;

    reg regwriteD, memtoRegD, memwriteD, branchD, aluSrcE, regdstE;
    reg [2:0] alucontrolE;
    reg [31:0] srcAE, writedataD, signimmE, pcplus4E;
    reg [4:0] rtE, rdE;

    wire regwriteE, memtoRegE, memwriteE, branchE, zeroE;
    wire [31:0] aluoutE, writedataE, pcbranchE;
    wire [4:0] writeregE;

    integer i;

    EX_Stage dut(
        regwriteD, memtoRegD, memwriteD, branchD,
        alucontrolE, aluSrcE, regdstE, srcAE,
        writedataD, rtE, rdE, signimmE, pcplus4E,
        regwriteE, memtoRegE, memwriteE, branchE,
        zeroE, aluoutE, writedataE, writeregE,
        pcbranchE
    );

    initial begin
        srcAE      = 32'h00001212;
        writedataD = 32'h00003434;
        signimmE   = 32'hfffffff;
        pcplus4E   = 32'h44444444;

        for (i = 0; i < 8; i = i + 1)
            testcase;
    end

    #10 $finish;

    task testcase;
    begin
        {regwriteD, memtoRegD, memwriteD, branchD,
         aluSrcE, regdstE} = $random;

        alucontrolE = i;

        case (alucontrolE)
            0: $display("ALU is performing AND");
            1: $display("ALU is performing OR");
            2: $display("ALU is performing ADD");
            6: $display("ALU is performing SUB");
            7: $display("ALU is performing SLT");
            default: $display("ALU Operation is invalid");
        endcase

        #1 $display("aluoutE = %h", aluoutE);

        if (zeroE != {aluoutE})
            $display("zero flag is malfunctioning");

        // verify control signals propagate unchanged
        if (regwriteE != regwriteD ||
            memtoRegE != memtoRegD ||
            memwriteE != memwriteD ||
            branchE   != branchD   ||
            writedataE != writedataD)
        begin
            $display("Control signals did not pass correctly!");
            $finish;
        end

        // verify branch adder
        if (pcbranchE != ((signimmE[29:0] << 2) + pcplus4E))
        begin
            $display("Branch Adder is malfunctioning");
            $finish;
        end

        if (i == 3)
            i = 5;
        end
    endtask
endmodule
```

Use of AI Tools:

ChatGPT Edu (GPT-5) was used to proofread the lab report for clarity and formatting.